

Entendendo o Serviço HTTP com Go (Golang)

Introdução

O **HTTP (HyperText Transfer Protocol)** é o protocolo responsável pela comunicação entre navegadores, aplicativos e servidores na Internet. Sempre que acessamos um site, fazemos uma consulta a uma API ou carregamos uma imagem em uma página, estamos utilizando o HTTP.

Na linguagem **Go (Golang)**, o pacote `net/http` oferece uma implementação completa para criação de servidores e clientes HTTP, permitindo desenvolver aplicações web de forma simples, rápida e eficiente.

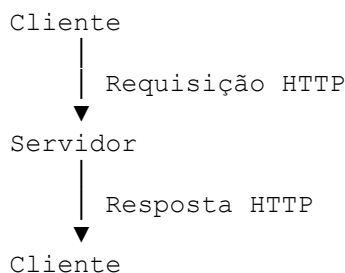
O que é HTTP?

HTTP é um protocolo de comunicação baseado no modelo **Cliente-Servidor**.

Neste modelo existem dois participantes principais:

- **Cliente:** quem realiza uma requisição (normalmente um navegador, aplicativo ou outro servidor).
- **Servidor:** quem recebe a requisição, processa e devolve uma resposta.

O fluxo é simples:



Principais características do HTTP

1. É baseado em requisições e respostas

Toda comunicação acontece através de uma requisição enviada pelo cliente e uma resposta enviada pelo servidor.

Exemplo:

```
Cliente → GET /produtos
Servidor → Lista de produtos
```

2. É Stateless

O HTTP não guarda informações entre uma requisição e outra.

Isso significa que cada nova requisição é completamente independente da anterior.

Por esse motivo utilizamos mecanismos como:

- Cookies
- Sessões
- Tokens JWT
- OAuth

para manter usuários autenticados.

3. Utiliza URLs

Cada recurso possui um endereço.

Exemplos:

`https://meusite.com`

`https://meusite.com/produtos`

`https://meusite.com/clientes/10`

Cada URL representa um recurso diferente.

4. Trabalha com Métodos HTTP

Cada requisição possui um verbo indicando sua finalidade.

Os mais comuns são:

Método	Finalidade
GET	Buscar informações
POST	Criar dados
PUT	Atualizar totalmente
PATCH	Atualizar parcialmente
DELETE	Remover dados

5. Possui Cabeçalhos (Headers)

Os cabeçalhos carregam informações importantes sobre a requisição.

Exemplos:

`Content-Type`

`Authorization`

`User-Agent`

`Accept`

`Cookie`

O servidor também envia cabeçalhos na resposta.

6. Utiliza Códigos de Status

Toda resposta possui um código indicando o resultado da operação.

Os principais são:

Código	Significado
200	OK
201	Criado
204	Sem conteúdo
400	Requisição inválida
401	Não autorizado
403	Proibido
404	Não encontrado
500	Erro interno do servidor

Estrutura de uma Requisição HTTP

Uma requisição possui quatro partes principais:

Método

URL

Cabeçalhos

Corpo (Body)

Exemplo:

```
GET /produtos HTTP/1.1
```

```
Host: localhost:8080
```

```
Accept: application/json
```

Estrutura de uma Resposta HTTP

O servidor responde com:

Status

Cabeçalhos

Corpo

Exemplo:

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
[  
  {  
    "id": 1,  
    "nome": "Notebook"  
  }  
]
```

Criando um Servidor HTTP em Go

Em Go, basta importar o pacote `net/http`.

```
package main

import (
    "fmt"
    "net/http"
)

func home(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Servidor funcionando!")
}

func main() {

    http.HandleFunc("/", home)

    fmt.Println("Servidor iniciado")

    http.ListenAndServe(":8080", nil)
}
```

Ao acessar

`http://localhost:8080`

o navegador receberá:

`Servidor funcionando!`

Entendendo o Handler

Um Handler é uma função responsável por atender uma rota.

Ela sempre recebe dois parâmetros:

```
func home(w http.ResponseWriter, r *http.Request)
```

ResponseWriter

Representa a resposta enviada ao cliente.

Tudo que for escrito nele aparecerá no navegador.

Exemplo:

```
fmt.Fprintln(w, "Olá Mundo")
```

Request

Representa toda a requisição recebida.

Ela contém:

- método
- URL
- parâmetros

- cabeçalhos
- corpo da requisição
- cookies

Exemplo:

```
fmt.Println(r.Method)
```

Criando várias rotas

```
package main

import (
    "fmt"
    "net/http"
)

func home(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Página Inicial")
}

func sobre(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Página Sobre")
}

func contato(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Página Contato")
}

func main() {

    http.HandleFunc("/", home)
    http.HandleFunc("/sobre", sobre)
    http.HandleFunc("/contato", contato)

    http.ListenAndServe(":8080", nil)
}
```

Agora temos:

localhost:8080/

localhost:8080/sobre

localhost:8080/contato

Trabalhando com métodos HTTP

Podemos verificar qual método foi utilizado.

```
func produtos(w http.ResponseWriter, r *http.Request) {

    if r.Method == "GET" {

        fmt.Fprintln(w, "Listando produtos")

    } else if r.Method == "POST" {

        fmt.Fprintln(w, "Criando produto")

    }
}
```

```
    } else {  
        http.Error(w, "Método não permitido", http.StatusMethodNotAllowed)  
    }  
}
```

Lendo parâmetros da URL

Se a URL for:

`http://localhost:8080?nome=Antonio`

podemos ler:

```
nome := r.URL.Query().Get("nome")  
  
fmt.Fprintln(w, nome)
```

Resultado:

Antonio

Enviando JSON

Uma API normalmente responde em JSON.

```
type Produto struct {  
    ID int `json:"id"`  
    Nome string `json:"nome"`  
}  
  
produto := Produto{  
    ID:1,  
    Nome:"Notebook",  
}  
  
w.Header().Set("Content-Type", "application/json")  
  
json.NewEncoder(w).Encode(produto)
```

Resposta:

```
{  
  "id":1,  
  "nome":"Notebook"  
}
```

Lendo JSON enviado pelo cliente

Quando o cliente envia um JSON:

```
{
  "id": 3,
  "nome": "Mouse"
}
```

podemos convertê-lo para uma estrutura Go:

```
var produto Produto

json.NewDecoder(r.Body).Decode(&produto)

fmt.Println(produto.Nome)
```

Definindo o código de resposta

```
w.WriteHeader(http.StatusCreated)
```

ou

```
http.Error(w, "Produto não encontrado", http.StatusNotFound)
```

Servindo arquivos estáticos

Também é possível disponibilizar imagens, arquivos CSS e JavaScript.

```
http.Handle("/static/",
    http.StripPrefix(
        "/static/",
        http.FileServer(http.Dir("static"))),
)
```

Assim, o conteúdo da pasta `static` ficará disponível pela URL:

```
/static/
```

Vantagens do pacote `net/http`

O pacote `net/http` oferece diversas vantagens:

- Faz parte da biblioteca padrão da linguagem.
- Não requer dependências externas.
- Alto desempenho.
- Excelente suporte à concorrência através das goroutines.
- API simples e intuitiva.
- Compatível com APIs REST e microserviços.
- Fácil integração com bancos de dados, autenticação e middlewares.

Conclusão

O protocolo HTTP é a base da comunicação na Web e está presente em praticamente todas as aplicações modernas. Compreender conceitos como requisições, respostas, métodos, cabeçalhos e códigos de status é essencial para qualquer desenvolvedor de aplicações web.

A linguagem Go simplifica significativamente o desenvolvimento de servidores HTTP por meio do pacote `net/http`, que fornece uma API robusta e eficiente. Com poucas linhas de código é possível criar servidores, definir rotas, manipular requisições, responder em JSON e construir APIs REST escaláveis. Essa simplicidade, aliada ao excelente desempenho e ao modelo de concorrência baseado em goroutines, faz do Go uma excelente escolha para o desenvolvimento de serviços web modernos.